



ENGINEERING FACULTY

Exam Practical Task

Course: **Computer Systems Structure**

Theme: **RISC Processor: AVR ASM**

Student: Igors Oļeņikovs

Student code: 93642

Group: 4501BTA

CONTENTS

Contents

1.	TASK DESCRIPTION	3
2.	PIN MAPPING	3
3.	IMPLEMENTATION SUMMARY	4
4.	PROGRAM DESIGN AND LOGIC	5
5.	KEY MECHANISM	9

1. TASK DESCRIPTION

The task requires creating a setup in the Wokwi simulation platform and writing a program in AVR Assembly for the Arduino Uno (ATmega328P). The program must drive five LEDs and one seven segment display, launched by a push button, without using any Arduino C/C++ functions - only direct AVR register and port manipulation

Seven segment display must show the numeric digits of the student code one by one, each for approximately one second. While a digit is shown, the LEDs must indicate whether the digit is odd (LEDs 1, 3, 5 ON) or even (LEDs 2, 4 ON). After all digits displayed, running light effect moves from LED1 to LED5 (0.5 s each) with the display blank. The system then returns to the initial alternating LED pattern and waits for the next button press.

Student code digits

Student code: 93642

Numeric sequence: 9 > 3 > 6 > 4 > 2

Step	Seven segment display	Digit type	LED output
1	9	Odd	LEDs 1, 3, and 5 ON
2	3	Odd	LEDs 1, 3, and 5 ON
3	6	Even	LEDs 2 and 4 ON
4	4	Even	LEDs 2 and 4 ON
5	2	Even	LEDs 2 and 4 ON

2. PIN MAPPING

The following table shows which AVR port and bit is connected to each hardware component. All five LEDs are on Port B, the seven-segment segments A - F are on Port C, segment G is on Port D, and the button uses Port D with the internal pull-up resistor enabled.

Component	AVR port / bit	Arduino pin	Direction
LED1 (odd)	PB5	13	Output
LED2 (even)	PB4	12	Output
LED3 (odd)	PB3	11	Output
LED4 (even)	PB2	10	Output
LED5 (odd)	PB1	9	Output
7-seg A	PC0	A0	Output
7-seg B	PC1	A1	Output
7-seg C	PC2	A2	Output
7-seg D	PC3	A3	Output

7-seg E	PC4	A4	Output
7-seg F	PC5	A5	Output
7-seg G	PD3	3	Output
Button	PD2	2	Input (pull-up)
7-seg COM	GND	—	Common cathode

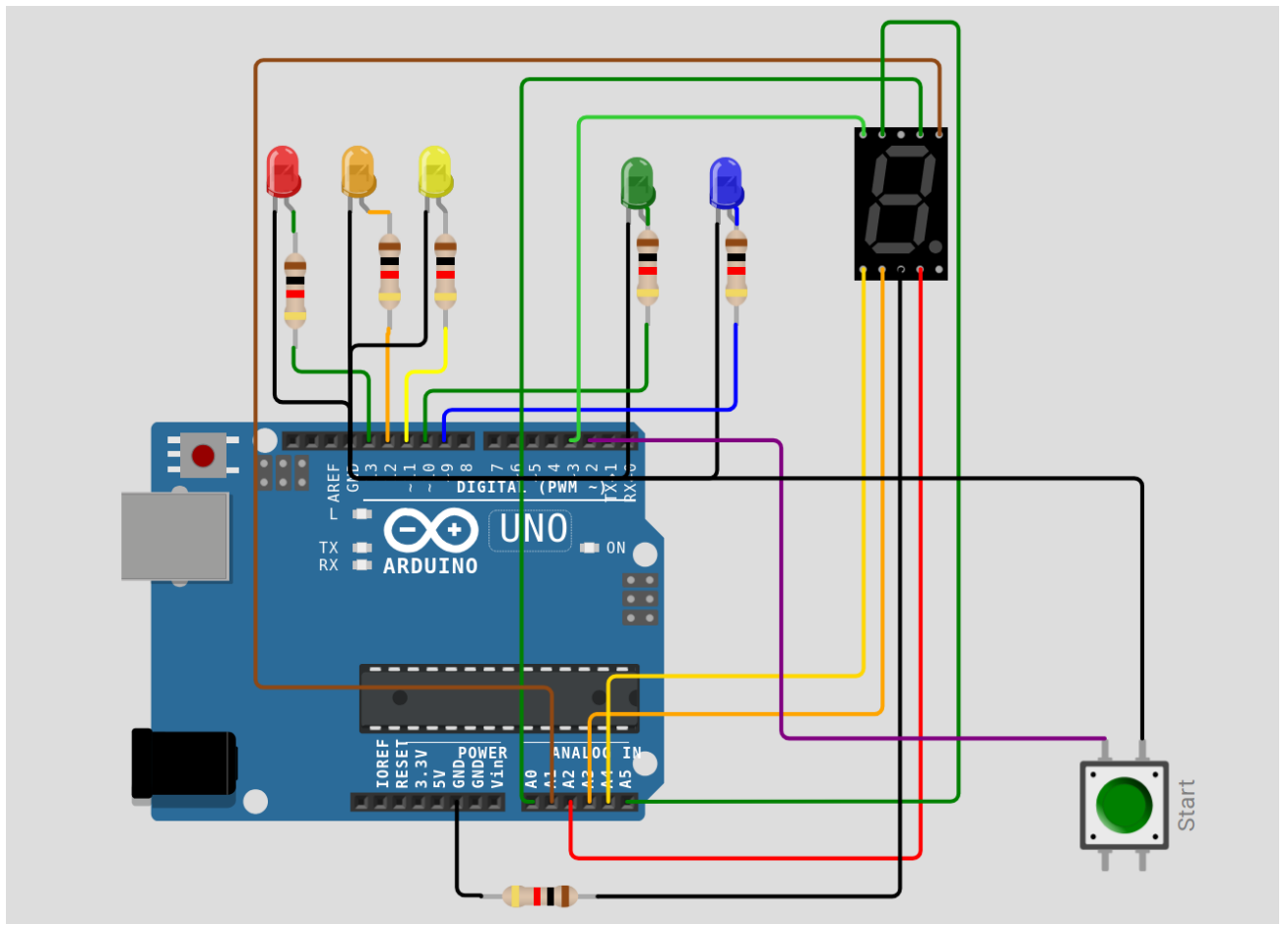


Fig.1 Wokwi circuit

3. IMPLEMENTATION SUMMARY

Five LEDs are connected to Arduino Uno pins 13 down to 9 through separate 1 kOhm resistors. These pins correspond to PB5, PB4, PB3, PB2 and PB1 on the ATmega328P. A common cathode seven segment display is driven from Port C (PC0 - PC5 for segments A - F) and PD3 for segment G. The common pin is connected to ground through a 1 kOhm resistor. A push button is connected to PD2 (pin 2) with the internal pull-up enabled, so the pin reads high when the button is released and low when it is pressed.

The program is organized into clearly separated sections: port initialization, the initial alternating LED state, button waiting with debounce, the student code display phase with odd/even LED selection, the running light finish phase and the return to the waiting state. Two lookup tables stored in program memory are read with the lpm instruction: one holds the seven segment patterns for digits 0 - 9 and the other holds the numeric digits of the student code in order.

4. PROGRAM DESIGN AND LOGIC

Port initialization

- DDRB = 0x3E configures PB5 - PB1 as LED outputs.
- DDRC = 0x3F configures PC5 - PC0 as outputs for segments A - F.
- PD3 is set as an output for segment G.
- PD2 is kept as an input and PORTD bit 2 is set high to enable the internal pull-up resistor on the button pin.

Initial state

- The seven segment display is blank (all segments off).
- PORTB is set to 0x2A, which turns on PB5, PB3 and PB1 - the alternating pattern LED1 ON, LED2 OFF, LED3 ON, LED4 OFF, LED5 ON.
- The program polls PD2 until the button is pressed.

Button launch

- The program waits for PD2 to go low (press), applies a 20 ms debounce delay, and confirms the press is still valid.
- It then waits for PD2 to return high (release) and applies another 20 ms debounce delay. This prevents a single long press from restarting the sequence multiple times.

Student code display phase

- A counter (r20) is set to 5, the number of digits in the student code.
- The Z register points to code_table in program memory. Each iteration loads the next digit with lpm r21, Z+.
- display_digit looks up the seven segment pattern in seg_table and writes the lower six bits to PORTC and bit 6 to PD3. The subroutine saves and restores Z internally so the caller's pointer is preserved
- The digit's least significant bit is tested with andi r22, 0x01: if set, the digit is odd and PORTB = 0x2A (LEDs 1, 3, 5) - if clear, the digit is even and PORTB = 0x14 (LEDs 2, 4).
- A ~1 sec delay is applied before moving to the next digit.

Running light finish phase

- The display is blank and a counter (r18) is set to 5.
- A single bit in r17 starts at 0x20 (LED1 / PB5) and is shifted right with lsr after each 0.5 s delay, lightening LED1 through LED5 in turn.
- After LED5 the program jumps back to initial state, restoring the alternating LED pattern and waiting for next button press

```
#define __SFR_OFFSET 0

#include "avr/io.h"

.global main

; LED patterns on PORTB bits PB5..PB1
; LED1=PB5, LED2=PB4, LED3=PB3, LED4=PB2, LED5=PB1
.equ LED_ODD,    0x2A    ; LEDs 1,3,5 ON
.equ LED_EVEN,   0x14    ; LEDs 2,4 ON
.equ LED_RUN_1,  0x20    ; running-light start at LED1
.equ DIGIT_COUNT, 5

main:
; Port initialization
ldi    r16, 0x3E
out    DDRB, r16
ldi    r16, 0x3F
out    DDRC, r16
sbi    DDRD, 3
cbi    DDRD, 2
sbi    PORTD, 2

; Initial state
initial_state:
call   display_blank
ldi    r16, LED_ODD
out    PORTB, r16

; Button waiting
wait_press:
sbis   PIND, 2
rjmp   button_pressed
rjmp   wait_press

button_pressed:
ldi    r25, hi8(20)
ldi    r24, lo8(20)
```

```

    call delay_ms
    sbis PIND, 2
    rjmp wait_release
    rjmp wait_press

wait_release:
    sbis PIND, 2
    rjmp wait_release
    ldi r25, hi8(20)
    ldi r24, lo8(20)
    call delay_ms

; Student code display phase
    ldi r20, DIGIT_COUNT
    ldi ZH, hi8(code_table)
    ldi ZL, lo8(code_table)

display_loop:
    lpm r21, Z+
    call display_digit
    mov r22, r21
    andi r22, 0x01
    breq even_digit
odd_digit:
    ldi r16, LED_ODD
    out PORTB, r16
    rjmp digit_hold
even_digit:
    ldi r16, LED_EVEN
    out PORTB, r16
digit_hold:
    ldi r25, hi8(1000)
    ldi r24, lo8(1000)
    call delay_ms
    dec r20
    brne display_loop

; Running light finish phase
    call display_blank
    ldi r18, DIGIT_COUNT
    ldi r17, LED_RUN_1
running_light:
    out PORTB, r17
    ldi r25, hi8(500)
    ldi r24, lo8(500)
    call delay_ms
    lsr r17

```

```

    dec    r18
    brne   running_light

; Return to waiting state
    rjmp   initial_state

; Subroutine: display_blank
; Turns off every segment of display
display_blank:
    ldi    r16, 0x00
    out    PORTC, r16
    cbi    PORTD, 3
    ret

; Subroutine: display_digit – show r21 (0-9) on 7-seg
; Saves and restores Z so callers Z is preserved
display_digit:
    push   r30
    push   r31
    ldi    ZH, hi8(seg_table)
    ldi    ZL, lo8(seg_table)
    add    ZL, r21
    clr    r19
    adc    ZH, r19
    lpm    r23, Z
    mov    r19, r23
    andi   r19, 0x3F
    out    PORTC, r19
    sbrc   r23, 6
    rjmp   seg_g_on
    cbi    PORTD, 3
    pop    r31
    pop    r30
    ret

seg_g_on:
    sbi    PORTD, 3
    pop    r31
    pop    r30
    ret

; Subroutine: delay_ms
delay_ms:
    ldi    r29, hi8(4000)
    ldi    r28, lo8(4000)
delay_inner:
    sbiw   r28, 1
    brne   delay_inner

```



```

    sbiw  r24, 1
    brne  delay_ms
    ret

; Seven-segment patterns for digits 0..9
; Bit mapping: bit0=A bit1=B bit2=C bit3=D bit4=E bit5=F bit6=G
seg_table:
    .byte 0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07, 0x7F, 0x6F
code_table:
    .byte 9, 3, 6, 4, 2

```

5. KEY MECHANISM

How the button is checked

The button is connected to PD2 with the internal pull-up enabled, so the pin is HIGH when the button is released and LOW when pressed. The program uses `sbis PIND, 2` to skip the next instruction when the pin is HIGH. A 20 ms debounce delay is applied both after detecting a press and after detecting a release. The program first waits for a press, then waits for a release, ensuring the main sequence launches exactly once per button cycle.

How the odd/even LED pattern is selected

After loading a digit from `code_table`, the program copies it into `r22` and executes `andi r22, 0x01` to isolate the least significant bit. If the bit is 1 the digit is odd and `PORTB` is loaded with `0x2A` (LEDs 1, 3, 5 ON). If the bit is 0 the digit is even and `PORTB` is loaded with `0x14` (LEDs 2, 4 ON). The `breq` instruction branches to the appropriate pattern.

How the delay timing is created

The `delay_ms` subroutine accepts a 16-bit millisecond count in `r25:r24`. For each millisecond it runs an inner loop of approximately 4000 iterations, each consuming four clock cycles (`sbiw + brne`). At 16 MHz, four cycles x 4000 = 16000 cycles = approx 1 ms. The outer loop decrements `r25:r24`. For the digit display phase, a value of 1000 is passed for ~1 second per digit. For the running light phase, a value of 500 is passed for ~0.5 second per step. The Y register (`r29:r28`) is used for the inner loop to avoid conflicts with the Z register.

How the seven segment digit patterns are selected

A lookup table `seg_table` in program memory stores one byte per digit (0-9). Each byte is a 7-bit pattern where bit 0 = segment A, bit 1 = B, bit 2 = C, bit 3 = D, bit 4 = E, bit 5 = F, and bit 6 = G. The `display_digit` subroutine calculates the address as `seg_table + r21`, reads the byte with `lpm`, writes bits 0-5 to `PORTC` (segments A-F), and tests bit 6 to control `PD3` (segment G). The subroutine preserves the caller's Z register by pushing it at entry and popping it on both return paths.

How the code table pointer is preserved

The `display_digit` subroutine internally modifies the Z register to access `seg_table`. To prevent this from corrupting the `code_table` pointer used by `display_loop`, `display_digit` saves `r30` (ZL) and `r31` (ZH) on the stack at entry and restores them before returning. Both the segment G-on and segment G-off return path include the `pop` instructions, ensuring correct Z restoration regardless of which path is taken. This allows `display_loop` to iterate through all five student code digits without any additional save/restore logic in the caller.

Video demonstration attached

Project saved at <https://wokwi.com/projects/467947593630873601>